You said:

exec(2).pyc
Code

Analise o arquivo em anexo passo a passo e determine ao final: este arquivo é benigno ou é um malware?

Worked for 50s

ChatGPT said:

Análise estática de malware — `exec(2).pyc`

MALWARE — BACKDOOR

Veredito: o arquivo apresenta comportamento malicioso. A análise do bytecode identificou a implementação de uma backdoor com funcionalidades de execução remota de comandos e transferência de arquivos.

Há, entretanto, erros de implementação que comprometem seu funcionamento completo.

1. Identificação do arquivo

Propriedade	Resultado
Arquivo	`exec(2).pyc`
Formato	Bytecode Python compilado
Linguagem	Python
Versão	CPython 3.13
Tamanho	3.990 bytes
Tipo	Timestamp-based bytecode
SHA-256	`25ed1fe44421faf1fecadc73aa8ed6fec4159f13fadd96fcce8877302a560e24`

O arquivo foi analisado sem executá-lo.

2. Bibliotecas importadas

O programa importa seis bibliotecas:

Biblioteca	Finalidade identificada
`socket`	Comunicação pela rede
`subprocess`	Execução de comandos do sistema
`json`	Serialização de dados
`os`	Operações no sistema operacional
`base64`	Codificação e decodificação de arquivos
`sys`	Controle da execução do programa

A combinação de `socket`, `subprocess`, `os` e `base64` é particularmente relevante porque permite implementar controle remoto de uma máquina.

3. Análise das funcionalidades

3.1. Estabelecimento de conexão remota

Foi identificada uma classe chamada `Backdoor`.

Seu construtor cria um socket TCP:

```
socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

Em seguida, conecta-se a um endereço IP e porta definidos diretamente no código:

```
192.168.43.38
4444
```

Isso significa que, ao iniciar, o programa tenta estabelecer uma conexão TCP com o endereço especificado.

Indicador suspeito

O comportamento é característico de uma backdoor com conexão de saída, também conhecida como *reverse connection*.

É importante destacar que `192.168.43.38` pertence a uma faixa de endereços privados. Não é possível determinar, apenas pelo arquivo, quem controla esse equipamento.

3.2. Execução remota de comandos

Foi encontrada a função:

`execute_system_commmand()`

Ela utiliza:

```
subprocess.check_output(
    command,
    shell=True
)
```

Essa função permite executar comandos do sistema operacional.

Na lógica do programa, o comando recebido pela conexão remota pode ser encaminhado para essa função.

Isso representa uma funcionalidade característica de um shell remoto.

3.3. Manipulação de diretórios

A função:

`change_working_directory_to()`

utiliza:

`os.chdir(path)`

Permite modificar o diretório de trabalho da máquina comprometida a partir de uma solicitação remota.

3.4. Leitura e transferência de arquivos

A função:

`read_file()`

abre arquivos em modo de leitura binária:

`open(path, 'rb')`

Depois, utiliza:

`base64.b64encode()`

Isso permite ler arquivos locais e preparar seu conteúdo para transferência pela conexão de rede.

Potencial de exfiltração

Essa funcionalidade pode permitir que um operador remoto obtenha arquivos da máquina.

3.5. Escrita e transferência de arquivos

A função:

`write_file()`

utiliza:

`open(path, 'wb')`

E posteriormente:

`base64.b64decode(content)`

Assim, permite gravar arquivos recebidos remotamente no computador.

Isso representa potencial de instalação de arquivos, substituição de conteúdo ou alteração de dados existentes.

3.6. Laço de controle remoto

A função `run()` implementa um mecanismo de recebimento e processamento de comandos.

Foram identificadas quatro operações especiais:

Comando	Comportamento
`exit`	Encerra a conexão e o programa
`cd`	Altera o diretório de trabalho
`download`	Lê arquivos locais
`upload`	Grava arquivos recebidos

Qualquer comando que não corresponda a essas operações especiais é encaminhado para a função de execução de comandos do sistema.

Isso demonstra que o objetivo aparente do programa é disponibilizar controle remoto sobre a máquina.

4. Análise dos problemas de implementação

Um aspecto importante é que o código apresenta erros que comprometem seu funcionamento.

Identifiquei três problemas relevantes:

Problema 1 — Recebimento de dados

A função `reliable_receive()` inicializa:

```
json_data = ''
```

Depois, tenta concatenar os dados recebidos pelo socket:

```
json_data += self.connection.recv(1024)
```

Entretanto, `recv()` retorna bytes, enquanto `json_data` é uma string.

Essa operação provoca um `TypeError` em Python 3, impedindo o processamento normal dos comandos.

Problema 2 — Envio de dados

A função `reliable_send()` utiliza:

```
json_data = json.dumps(data)
self.connection.send(json_data)
```

`json.dumps()` retorna uma string, mas `socket.send()` exige um objeto do tipo bytes.

Portanto, essa operação também apresenta incompatibilidade de tipos.

Problema 3 — Ausência de tratamento adequado de mensagens

O recebimento não implementa corretamente a decodificação e desserialização JSON. Além disso, seu laço não apresenta uma condição funcional para concluir normalmente o recebimento.

Esses problemas indicam que o código provavelmente está incompleto ou foi implementado incorretamente.

Consequência: embora apresente características inequívocas de uma backdoor, não consegui confirmar que todas as funcionalidades de controle remoto seriam operacionalmente utilizáveis nessa versão.

5. Indicadores de comprometimento (IOCs)

Os principais indicadores identificados são:

Indicador	Valor
Endereço IP configurado	`192.168.43.38`
Porta TCP	`4444`
Classe principal	`Backdoor`
Execução de comandos	`subprocess.check_output`
Comunicação de rede	`socket`
Leitura de arquivos	`open(path, 'rb')`
Escrita de arquivos	`open(path, 'wb')`
Codificação	Base64

Não foram identificados, nessa análise, mecanismos evidentes de persistência, como inicialização automática, modificação de registro ou criação de tarefas agendadas.

6. Veredito final

CLASSIFICAÇÃO: MALWARE

BACKDOOR

Categoria secundária: RAT (*Remote Access Trojan*) / Reverse Shell.

Justificativa: o programa estabelece uma conexão remota com um endereço IP predefinido e implementa funcionalidades para execução de comandos, leitura de arquivos e gravação de arquivos remotamente.

Ressalva técnica: existem erros de programação que impedem o funcionamento normal de partes importantes da comunicação. Isso não torna o código benigno; indica que a implementação da backdoor está defeituosa.

Conclusão: o arquivo `exec(2).pyc` deve ser classificado como **malware da categoria Backdoor**, com características de um Trojan de Acesso Remoto (RAT). Sua finalidade aparente é permitir controle remoto e manipulação de arquivos na máquina vítima.

Recomendação: não execute esse arquivo em seu computador. Caso ele já tenha sido executado, investigue eventuais conexões com `192.168.43.38:4444` e isole a máquina até esclarecer sua origem.

You said:

Qual é a categoria principal deste malware?

ChatGPT said:

A categoria principal deste malware é **Backdoor (porta dos fundos)**.

Mais especificamente, apresenta características de um **RAT (Remote Access Trojan)**, permitindo execução remota de comandos e manipulação de arquivos na máquina vítima.